



C3

In-context learning & RAG

20 minutes

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Explain why teams retrieve context instead of retraining — and walk the embed → retrieve → generate workflow you're about to build.

Fine-tune or retrieve?

Two ways to make a generic model use your private data.

Fine-tune

Change the model's weights on your examples.

- Teaches style / format / behaviour
- Stable domains; costly to update
- Needs many repetitions of a fact

Retrieve (RAG)

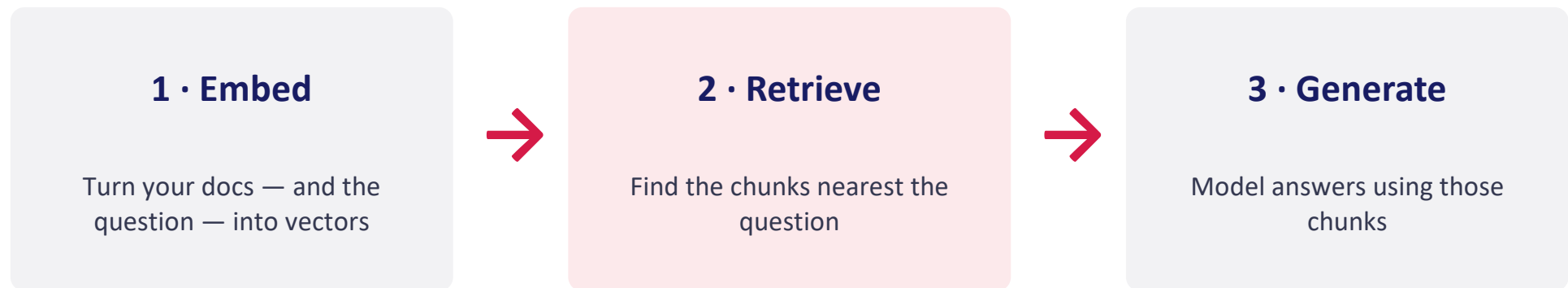
Fetch relevant text at query time; put it in the prompt.

- Supplies facts & current context
- Private, changing, large corpora
- Update = change the data, not the model

Takeaway Rule of thumb: retrieve for facts, fine-tune for behaviour — often both.

The RAG workflow — three stages

Everything the Colab does, in one line: embed → retrieve → generate.



Offline: chunk → embed → build the index. Online: stages 1→2→3 run on every question.

For the builders: chunk → embed → index (offline) · embed query → ANN top-k → assemble prompt → generate (online)

Takeaway RAG = embed → retrieve → generate. That's the whole idea.

Embeddings & similarity, without the maths

How does a machine know two sentences 'mean' the same thing?

Text → numbers

"cancel my card" → [0.12, -0.41, ...]
"block my card" → [0.13, -0.39, ...]

Different words, almost the same vector.

Nearest = most relevant

- Embed the user's question
- Find the closest chunks in the store
- Those chunks go into the prompt

Takeaway Meaning becomes distance; retrieval becomes 'find the nearest'.

Under the hood — vectors, similarity & the index

FOR BUILDERS

For the builders: how 'find the nearest meaning' actually works at scale (skim for a non-technical room).

1 · Embedding = a coordinate for meaning

- Text / image → a fixed-length vector ($\approx 1,536$ numbers)
- Trained so similar meaning → nearby points
- Similarity = cosine (angle), not shared words
- No single number means anything — only distance does

2 · Vector index = nearest-neighbour, fast

- Brute force: compare all N vectors — exact but slow
- ANN index: HNSW (shortcut graph) or IVF (clusters)
- Approximate → top-k matches in milliseconds at scale
- Built OFFLINE once; queried ONLINE per question

Offline docs → embed → build index **Online** question → embed → ANN search → top-k → prompt

Takeaway Embeddings turn meaning into coordinates; the index makes them searchable — fast.

The knobs you'll actually turn

In the Colab you change these and watch the answer change.

Chunking

How you cut documents before embedding.

- Too big: noisy, wastes tokens
- Too small: loses context
- Overlap keeps ideas intact

Top-k retrieval

How many chunks you fetch per question.

- Too few: misses the answer
- Too many: dilutes + costs tokens
- Tune k against your evals

Takeaway Chunking and k are the basics — the advanced levers (re-ranking, hybrid, query transforms) come next.

Beyond naive RAG — the advanced levers

For the builders (skim for a non-technical room): what you reach for when chunking + top-k aren't enough.

Re-ranking — retrieve wide, then a cross-encoder re-scores (query, chunk) pairs — keep the best few

Hybrid search — dense (meaning) + sparse (BM25 keyword) results, fused with RRF

Query transforms — rewrite · multi-query · HyDE — fix vague or mismatched questions

Metadata filters — pre-filter by tenant / date / type / access before the vector search

Chunking & structure — recursive/semantic · parent-document · contextual retrieval · GraphRAG

Terms: cross-encoder = scores a (query, chunk) pair together (accurate, slow) · BM25 = classic keyword ranking · RRF = Reciprocal Rank Fusion (merge ranked lists) · HyDE = embed a hypothetical answer · contextual retrieval = prepend doc context to each chunk · GraphRAG = retrieve over a knowledge graph

Takeaway When basic RAG stalls, re-ranking and hybrid search are the first two upgrades — measure each against your evals.

Who you'd buy from — by layer

Illustrative, not endorsements — global and Asia side by side.

Layer	Global	Asia-based / preferred
Compute / serving	AWS · Azure · GCP · NVIDIA · vLLM, Modal	Alibaba Cloud · Huawei Cloud · Tencent Cloud
Data / ETL	Databricks · Snowflake · Airflow	Alibaba MaxCompute · Huawei Cloud
Embeddings	OpenAI · Cohere · Voyage	BGE (BAAI) · Qwen · Jina AI
Vector store	Pinecone · Weaviate · Qdrant · pgvector	Milvus / Zilliz · Tencent · Alibaba AnalyticDB
Orchestration	LangChain · LlamaIndex	Dify · LlamaIndex
Model APIs	OpenAI · Anthropic · Google · Llama, Mistral	Qwen · DeepSeek · GLM/Zhipu · ERNIE · Kimi; IN: Sarvam
Observability / evals	LangSmith · Langfuse · Arize · W&B	Langfuse (OSS, self-host)

Takeaway Almost every layer is a buy decision — the skill is choosing, not building.

When to RAG — and when not to

The routing decision: don't reach for RAG reflexively.

Situation	Reach for
Private / changing / large knowledge; need citations	RAG
Need a fixed style, format, or behaviour	Fine-tune
The whole corpus fits in the prompt	Long context, skip retrieval
A structured lookup or metric	A database / SQL — no LLM
A simple, stable rule or mapping	A narrow model or code (recall C1)
Needs actions or multi-step reasoning	Agents + tools (Class 4)

Takeaway RAG is one tool. Knowing when NOT to use it is the senior skill.

RAG in the wild

Same pattern, different corpora — plus where to skip it.

Banking

DBS EKB

Q&A over internal policy + product docs

Grounded, auditable answers

Employee-facing first

Healthcare

illustrative

Answers from a hospital's own protocols

Cites the source guideline

Human-in-the-loop

Support / Legal

illustrative

Help-centre + customer history

Contract / precedent retrieval

Cheap to keep current

Takeaway If the value is 'answer from OUR documents', it's probably RAG.

Where RAG breaks

Most 'the model is wrong' bugs are actually retrieval bugs.

Bad chunks — split mid-idea, so no chunk holds the answer

Wrong retrieval — nearest $\neq$ relevant; the query was ambiguous

Context overflow — right chunk retrieved, then truncated out

Stale index — source changed; embeddings never refreshed

Measure it, don't vibe it: retrieval — recall@k (right chunk in top-k?) · precision@k (how little noise) · MRR / nDCG (ranked high?) · answer — faithfulness / groundedness (supported by the chunks?) → C4

Takeaway Debug retrieval first — and measure it (recall@k), don't guess. The model is the last suspect.

QUICK CHECK 1 • WOOC LAP

In a RAG system, what sits between the user's question and the model?

- A) A bigger model
- B) More training
- C) Retrieval — find the relevant chunks first
- D) A longer prompt template

Vote in Wooclap — answer on the next slide.

QUICK CHECK 1 · ANSWER

C — retrieval.

Retrieval fetches the relevant chunks and injects them into the prompt — the core of RAG (and what the Colab builds).

QUICK CHECK 2 • WOOC LAP

In RAG, what does the embedding model produce?

A) A vector — numbers that capture meaning

B) A short summary of the text

C) The final answer to the user

D) A keyword index

Vote in Wooclap — answer on the next slide.

QUICK CHECK 2 · ANSWER

A — a vector: numbers that capture meaning.

Embeddings turn meaning into coordinates; the vector store finds the nearest ones (slides 4–5).

QUICK CHECK 3 • WOOC LAP

A RAG answer is fluent, confident, and wrong — with a real-looking citation. What do you check FIRST?

A) Swap in a bigger model

B) Fine-tune on more data

C) Lengthen the system prompt

D) Retrieval — chunking + what got retrieved

Vote in Wooclap — answer on the next slide.

QUICK CHECK 3 · ANSWER

D — debug retrieval first.

Fluent-but-wrong usually means the right chunk wasn't retrieved. The model is the last suspect (slide 10).

QUICK CHECK 4 • WOOC LAP

5 million chunks, and you need retrieval under 100 ms. Best approach?

- A) Brute-force cosine over all 5M each query
- B) Paste all 5M chunks into the prompt
- C) An approximate nearest-neighbour index (HNSW / IVF)
- D) Fine-tune the model on all 5M chunks

Vote in Wooclap — answer on the next slide.

QUICK CHECK 4 · ANSWER

C — an ANN index (HNSW / IVF).

Approximate nearest-neighbour = milliseconds at scale. Brute force is too slow; long context and fine-tune don't solve retrieval (slide 5).

QUICK CHECK 5 • WOOC LAP

Embeddings look fine, but the TOP retrieved chunk still isn't the most relevant. Best first upgrade?

A) Add a re-ranker: retrieve wide, re-score, keep the best few

B) A bigger embedding model

C) Just raise top-k to 50

D) Switch to fine-tuning

Vote in Wooclap — answer on the next slide.

QUICK CHECK 5 · ANSWER

A — add a re-ranker.

Retrieve wide, re-score with a cross-encoder, keep the best few — the first upgrade when basic retrieval plateaus.

QUICK CHECK 6 • WOOC LAP

You tuned chunking and want to PROVE retrieval got better. Which signal is most direct?

- A) The answer 'reads better' to you
- B) Total token cost per query
- C) The model's public benchmark score
- D) recall@k — did the right chunk make the top-k?

Vote in Wooclap — answer on the next slide.

QUICK CHECK 6 · ANSWER

D — recall@k.

Did the right chunk make the top-k? The direct measure of a retrieval change (slide 11 → C4 evals).

The build — what we're about to do

Before any code: a tiny RAG over a small doc set, so every stage you just learned is visible.

What you'll build

A minimal RAG pipeline.

- Chunk + embed a few docs → index
- Ask a question → retrieve top-k
- Generate a grounded, cited answer
- Turn chunking / k and watch quality move

How we'll run it

~25 min — follow along or just watch.

- Notebook A: token-free LOCAL (no API key)
- Notebook B: API version (best-in-class)
- Pre-run HTML fallback if IT fails
- We'll break retrieval on purpose

Takeaway You just learned the map — now watch every stage run end-to-end.

C3 · TAKEAWAY → TO THE COLAB

RAG = embed → retrieve → generate. The model is rented; the retrieval is yours.

Now we build it — twice: a token-free local pipeline, then the API version.